

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проектированию
по курсу «Логика и основы алгоритмизации в
инженерных задачах»
на тему «Реализация алгоритма Дейкстры»

25.12.25
отметка
[подпись]

Выполнил:
Студент группы 24ВВВ3
Макунькин А.М.

Принял:
к.т.н. доцент Юрова О.В.

Пенза 2025

ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Факультет Вычислительной техники

Кафедра "Вычислительная техника"

"УТВЕРЖДАЮ"

Зав. кафедрой ВТ Михрохин М.В.

« » 20

ЗАДАНИЕ

на курсовое проектирование по курсу

Логика и основы алгоритмизации вычислительных задач
Студенту Михрохину Виталию Сергеевичу Группа 24ВВВВЗ
Тема проекта Реализация алгоритма Дейкстры

Исходные данные (технические требования) на проектирование

- Разработка алгоритмов и программного обеспечения в соответствии с данным заданием курсового проекта.
Техническое задание должно содержать:
1. Постановку задачи;
 2. Теоретическую часть задания;
 3. Описание алгоритма поставленной задачи;
 4. Пример ручного расчета задачи и включения (на небольшом участке работы алгоритма);
 5. Описание самой программы;
 6. Тесты;
 7. Список литературы;
 8. Метрики программы;
 9. Результаты работы программы.

Объем работы по курсу

1. Расчетная часть

Ручной расчет работоспособности

2. Графическая часть

Схема алгоритма в форме блок-схем.

3. Экспериментальная часть

Тестирование программы;

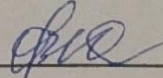
Результаты работы программы на тестовых данных

Срок выполнения проекта по разделам

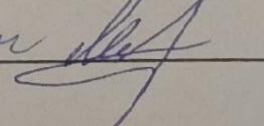
- 1 Исследование теоретической части курсового
- 2 Разработка алгоритмов программирования
- 3 Разработка программы
- 4 Тестирование и завершение разработки программы
- 5 Оформление пояснительной записки
- 6
- 7
- 8

Дата выдачи задания "12" 09.2025

Дата защиты проекта " " "

Руководитель Нарова Ольга Викторовна 

Задание получил "12" сентября 2025 г.

Студент Макушкин Юрий Маркович 

Содержание

Реферат	5
Введение	6
1. Постановка задачи	7
2. Теоретическая часть задания	8
3. Описание программы.....	10
3.1. Структура Graph	10
3.2. Функция <code>dijkstra()</code>	14
4. Тестирование	21
4.1. Создание графа (последовательности)	24
4.2. Просмотр созданного графа (последовательности)	25
4.3. Анализ созданного графа (последовательности)	25
4.4. Сохранение результатов.....	27
4.5. Результаты	29
5. Ручной расчет	30
5.1. Пошаговый расчет алгоритма	30
Заключение.....	33
Список литературы	34
Приложение А.....	35

Реферат

Отчет содержит: 54 страницы, 21 рисунок, 2 таблицы, 5 источников.

АЛГОРИТМ ДЕЙКСТРЫ, ВЗВЕШЕННЫЙ ГРАФ, МАТРИЦА СМЕЖНОСТИ, ОРИЕНТИРОВАННЫЙ ГРАФ, НЕОРИЕНТИРОВАННЫЙ ГРАФ, КРАТЧАЙШИЙ ПУТЬ, РЕЛАКСАЦИЯ РЕБЕР, ИЗОЛИРОВАННЫЕ ВЕРШИНЫ, ПЕТЛИ, ДОСТИЖИМОСТЬ, ПОИСК В ШИРИНУ, ДИНАМИЧЕСКОЕ ПРОГРАММИРОВАНИЕ, ЖАДНЫЙ АЛГОРИТМ.

Цель исследования – разработка программы, реализующей алгоритм Дейкстры для нахождения кратчайших путей во взвешенных графах с неотрицательными весами ребер, с поддержкой различных типов графов и параметров генерации.

В работе рассмотрены принципы работы алгоритма Дейкстры, включая стратегию "жадного" выбора и использование динамического программирования для накопления оптимальных расстояний. Установлено, что алгоритм эффективно решает задачу поиска кратчайших путей из одной вершины во все остальные для графов с неотрицательными весами, обеспечивая оптимальное время работы при использовании соответствующих структур данных. Реализация включает поддержку как ориентированных, так и неориентированных графов, возможность генерации графов с заданными параметрами, обработку изолированных вершин и петель, а также пошаговую визуализацию работы алгоритма для графов малого размера.

Введение

Алгоритм Дейкстры (англ. Dijkstra's algorithm) — это классический алгоритм на графах, разработанный нидерландским ученым Эдсгером Дейкстрой в 1959 году. Он предназначен для нахождения кратчайших путей от заданной начальной вершины до всех остальных вершин во взвешенном графе с неотрицательными весами ребер.

Основное отличие алгоритма Дейкстры от других алгоритмов поиска пути (таких как поиск в ширину) заключается в его способности корректно работать с взвешенными графами, где каждое ребро имеет определенную стоимость или длину. В то время как поиск в ширину находит кратчайшие пути в терминах количества ребер, алгоритм Дейкстры минимизирует сумму весов ребер вдоль пути.

В качестве среды разработки была выбрана среда Microsoft Visual Studio, язык программирования — Си. Этот выбор обусловлен требованием к производительности и низкоуровневому контролю над памятью, что важно для эффективной реализации алгоритмов на графах, особенно при работе с большими наборами данных.

Целью данной работы является разработка программы на языке Си, которая реализует алгоритм Дейкстры для нахождения кратчайших путей во взвешенных графах различных типов (ориентированных и неориентированных). Программа предусматривает генерацию графов с заданными параметрами, пошаговую визуализацию работы алгоритма для учебных целей и сохранение результатов работы для последующего анализа.

1. Постановка задачи

Требуется разработать программу, реализующую алгоритм Дейкстры для нахождения кратчайших путей из заданной начальной вершины во все остальные вершины взвешенного графа.

Исходный граф в программе должен задаваться матрицей смежности с весами ребер, причём при генерации данных должны быть предусмотрены различные настройки и граничные условия. Программа должна предоставлять пользователю возможность задания параметров графа: количества вершин (от 1 до 100), типа графа (ориентированный или неориентированный), разрешения или запрета петель у вершин, разрешения или запрета изолированных вершин.

После генерации данных на экран должна выводиться матрица смежности графа, где ненулевые значения представляют веса соответствующих ребер. Пользователь должен иметь возможность указать стартовую вершину, из которой будет осуществляться поиск кратчайших путей.

Для графов небольшого размера (до 10 вершин) программа должна обеспечивать пошаговую демонстрацию работы алгоритма с отображением текущих расстояний до вершин, состояния посещенных вершин и процесса обновления оценок кратчайших путей. Для графов большего размера должен выводиться только финальный результат в виде таблицы, содержащей для каждой вершины кратчайшее расстояние от начальной вершины и сам путь.

Программа должна корректно обрабатывать все возможные сценарии, включая наличие недостижимых вершин, изолированных компонент связности и графов с различной плотностью ребер. Все результаты работы алгоритма должны автоматически сохраняться в текстовый файл для последующего анализа.

2. Теоретическая часть задания

Алгоритм Дейкстры (рисунок 1) является одним из фундаментальных алгоритмов теории графов, предназначенным для решения задачи поиска кратчайшего пути от заданной начальной вершины до всех остальных вершин во взвешенном графе с неотрицательными весами ребер. Алгоритм был разработан нидерландским ученым Эдсгером Дейкстрой в 1959 году и с тех пор нашел широкое применение в различных областях, включая сетевые технологии, транспортное планирование и геоинформационные системы.

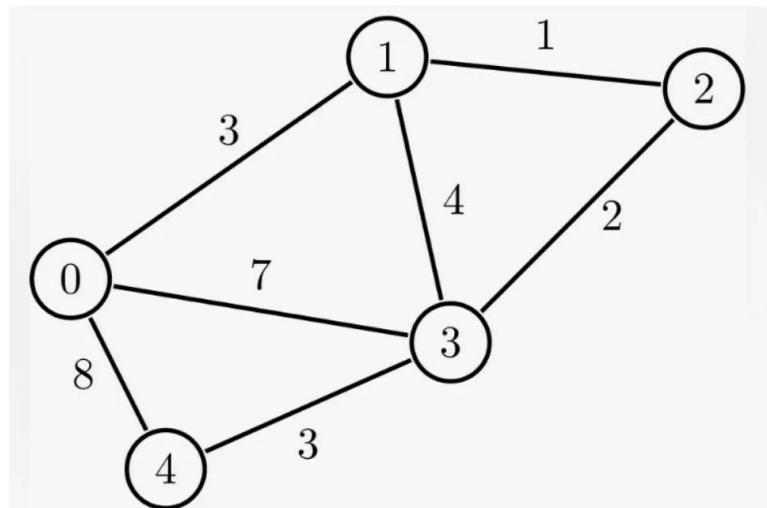


Рисунок 1 – Пример взвешенного графа

Основная идея алгоритма Дейкстры основана на принципе динамического программирования и стратегии "жадного" выбора. Алгоритм поддерживает множество посещенных вершин S и массив кратчайших расстояний $d[v]$ от начальной вершины s до каждой вершины $v \in V$. На каждом шаге выбирается вершина $u \notin S$ с минимальным значением $d[u]$, которая добавляется в множество S , после чего выполняется релаксация всех исходящих из u ребер: для каждого ребра (u, v) с весом $w(u, v)$ проверяется, можно ли улучшить расстояние до вершины v , используя путь через u : если $d[u] + w(u, v) < d[v]$, то значение $d[v]$ обновляется.

Формальное описание алгоритма:

Инициализация: для всех вершин $v \in V$ установить $d[v] = \infty$, для начальной вершины s установить $d[s] = 0$

Создать множество непосещенных вершин $Q = V$

Пока Q не пусто:

а) Найти вершину $u \in Q$ с минимальным значением $d[u]$

б) Удалить u из Q

в) Для каждого соседа v вершины u :

Если $d[u] + w(u, v) < d[v]$, то установить $d[v] = d[u] + w(u, v)$

Возвратить массив d

Важным свойством алгоритма Дейкстры является его корректность для графов с неотрицательными весами ребер. При наличии отрицательных весов алгоритм может давать некорректные результаты, так как предположение о том, что найденный кратчайший путь до вершины u является окончательным, перестает выполняться.

Для эффективной реализации алгоритма критически важным является способ выбора вершины с минимальным расстоянием на шаге 3а. При использовании простого линейного поиска сложность алгоритма составляет $O(V^2 + E)$, где V — количество вершин, E — количество ребер. Однако при использовании приоритетной очереди (например, на основе двоичной кучи) сложность уменьшается до $O((V + E) \log V)$, что делает алгоритм эффективным для разреженных графов.

3. Описание программы

Программа была написана в среде разработки Visual Studio 2022 на языке C с использованием процедурного подхода программирования. Реализация выполнена в виде консольного приложения, обеспечивающего генерацию взвешенных графов, их визуализацию и выполнение алгоритма Дейкстры для нахождения кратчайших путей.

Далее приведено описание структуры данных, функций и алгоритмов, используемых в программе. Полный листинг программы представлен в Приложении А.

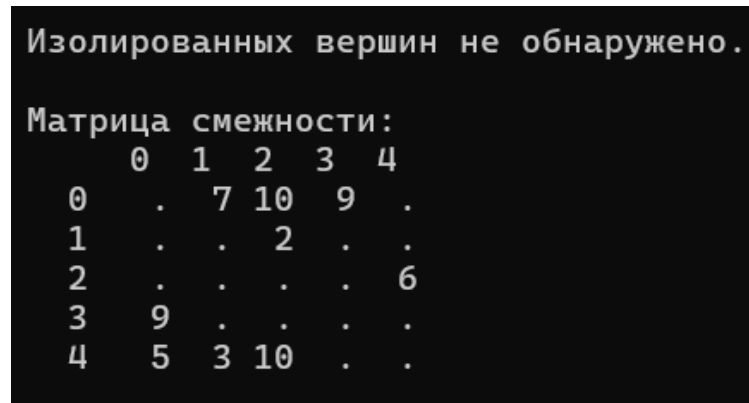
3.1. Структура Graph

Для хранения графа используется структура данных, содержащая матрицу смежности, количество вершин и флаги, определяющие свойства графа (ориентированность, разрешение петель, разрешение изолированных вершин). Эта структура позволяет удобно передавать данные между функциями программы. Программа начинается с запроса у пользователя параметров графа: количества вершин, типа графа, разрешения петель и изолированных вершин. После ввода этих данных создаётся граф с помощью функции `createGraph`, которая выделяет память под матрицу смежности и инициализирует её нулями (Рисунок 2).

```
=== РЕАЛИЗАЦИЯ АЛГОРИТМА ДЕЙКСТРЫ ===  
  
Введите количество вершин (1-100): 5  
  
Тип графа:  
1. Ориентированный  
2. Неориентированный  
Выберите (1-2): 1  
  
Разрешить петли у вершин? (0-нет, 1-да): 0  
  
Разрешить изолированные вершины? (0-нет, 1-да): 0
```

Рисунок 2 – Меню программы с выбором вида графа и его типов

Затем программа заполняет граф случайными данными в функции generateGraph. Здесь учитываются все заданные параметры: вероятность создания рёбер, тип графа, возможность петель и изолированных вершин. Особое внимание уделяется проверке изолированных вершин — если пользователь запретил их, но они появились при генерации, программа автоматически добавляет рёбра, чтобы все вершины были связаны (Рисунок 3).



Изолированных вершин не обнаружено.

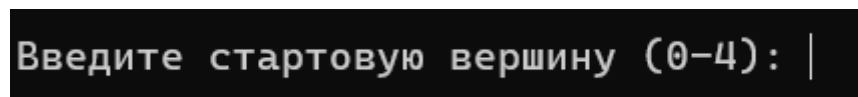
Матрица смежности:

	0	1	2	3	4
0	.	7	10	9	.
1	.	.	2	.	.
2	.	.	.	.	6
3	9	.	.	.	.
4	5	3	10	.	.

Рисунок 3 – Матрица смежности с отслеживанием изолированных вершин

После генерации графа программа выводит его матрицу смежности в виде удобной таблицы, где ненулевые значения представляют веса рёбер, а нули отображаются точками для лучшей читаемости. Пользователь видит структуру созданного графа и может оценить его сложность.

Далее программа запрашивает стартовую вершину для алгоритма Дейкстры (Рисунок 4).



Введите стартовую вершину (0-4): |

Рисунок 4 – Выбор стартовой вершины

В зависимости от размера графа выбирается режим работы: для графов до 10 вершин включается пошаговая реализация, где показывается каждый шаг алгоритма, а для больших графов выводится только итоговая таблица результатов (Рисунок 5,6).

Количество вершин ≤ 10 , будет выполнена пошаговая демонстрация
Нажимайте Enter после каждого шага для продолжения...

Рисунок 5 – предупреждение о меньшем количестве вершин

```
=== НАЧАЛО АЛГОРИТМА ДЕЙКСТРЫ (стартовая вершина: 0) ===

--- Шаг 1 ---
Текущие расстояния:   0  INF  INF  INF  INF
Посещенные вершины:  нет
Выбираем вершину 0 (минимальное расстояние: 0)
  Вершина 0 уже посещена
  Проверяем переход 0->1 (вес 7): новое расстояние 7 < INF – ОБНОВЛЯЕМ
  Проверяем переход 0->2 (вес 10): новое расстояние 10 < INF – ОБНОВЛЯЕМ
  Проверяем переход 0->3 (вес 9): новое расстояние 9 < INF – ОБНОВЛЯЕМ
  Переход 0->4: ребра нет

Нажмите Enter для продолжения...

--- Шаг 2 ---
Текущие расстояния:   0   7  10   9  INF
Посещенные вершины:  0
Выбираем вершину 1 (минимальное расстояние: 7)
  Вершина 0 уже посещена
  Вершина 1 уже посещена
  Проверяем переход 1->2 (вес 2): новое расстояние 9 < старого 10 – ОБНОВЛЯЕМ
  Переход 1->3: ребра нет
  Переход 1->4: ребра нет

Нажмите Enter для продолжения...

--- Шаг 3 ---
Текущие расстояния:   0   7   9   9  INF
Посещенные вершины:  0 1
Выбираем вершину 3 (минимальное расстояние: 9)
  Вершина 0 уже посещена
  Вершина 1 уже посещена
  Переход 3->2: ребра нет
  Вершина 3 уже посещена
  Переход 3->4: ребра нет

Нажмите Enter для продолжения...

--- Шаг 4 ---
Текущие расстояния:   0   7   9   9  INF
Посещенные вершины:  0 1 3
Выбираем вершину 2 (минимальное расстояние: 9)
  Вершина 0 уже посещена
  Вершина 1 уже посещена
  Вершина 2 уже посещена
  Вершина 3 уже посещена
  Проверяем переход 2->4 (вес 6): новое расстояние 15 < INF – ОБНОВЛЯЕМ

Нажмите Enter для продолжения...

--- Шаг 5 ---
Текущие расстояния:   0   7   9   9  15
Посещенные вершины:  0 1 2 3
Выбираем вершину 4 (минимальное расстояние: 15)
  Вершина 0 уже посещена
  Вершина 1 уже посещена
  Вершина 2 уже посещена
  Вершина 3 уже посещена
  Вершина 4 уже посещена
```

Рисунок 6 – Пошаговая реализация

После того, как была завершена пошаговая реализация, выводится таблица с результатами, где отображена стартовая вершина, расстояние и путь до остальных вершин (Рисунок 7).

=== РЕЗУЛЬТАТЫ АЛГОРИТМА ДЕЙКСТРЫ ===

Вершина	Расстояние	Путь
0	0	0
1	7	0->1
2	9	0->1->2
3	9	0->3
4	15	0->1->2->4

Рисунок 7 – Вывод таблицы результата

После того, как программа была завершена, выводится строка, которая дает знать о том, что данный результат был сохранен в виде файла и дается выбор, можно ли начать выполнение программы заново. Сохранение файла происходит автоматически, но если пользователь решит выполнить программу заново, то прошлый результат в файле будет стерт и перезапишет новый (Рисунок 8).

Результаты сохранены в файл 'dijkstra_results.txt'

Рисунок 8 – Сохранение файла

В ином случае, если пользователь введёт количество вершин, превышающий минимум, а именно больше 10 вершин, то программа предупредит об этом и затем автоматически пропустит выполнение пошаговой реализации, сразу переходя к выводу таблицы результата (Рисунок 9).

Количество вершин > 10, выводится только итоговая таблица

=== НАЧАЛО АЛГОРИТМА ДЕЙКСТРЫ (стартовая вершина: 1) ===

=== РЕЗУЛЬТАТЫ АЛГОРИТМА ДЕЙКСТРЫ ===

Вершина	Расстояние	Путь
0	4	1->0
1	0	1
2	9	1->0->2
3	21	1->0->2->8->3
4	16	1->0->2->4
5	8	1->5
6	15	1->5->9->6
7	10	1->7
8	11	1->0->2->8
9	11	1->5->9
10	24	1->0->2->4->10

Результаты сохранены в файл 'dijkstra_results.txt'

Рисунок 9 – Вывод таблицы без пошаговой реализации

3.2. Функция `dijkstra()`

Данная функция отвечает за непосредственную реализацию алгоритма Дейкстры для нахождения кратчайших путей от заданной стартовой вершины до всех остальных вершин графа. Функция принимает три параметра: указатель на структуру `Graph` (содержащую матрицу смежности и параметры графа), целое число `start` (индекс стартовой вершины) и целое число `stepByStep` (флаг, определяющий режим пошагового выполнения).

ФУНКЦИЯ `dijkstra(graph, start, stepByStep)`:

// Инициализация массивов

`dist` = массив размером `graph->vertices`, заполненный `INF`

`visited` = массив размером `graph->vertices`, заполненный `false`

`parent` = массив размером `graph->vertices`, заполненный `-1`

`dist[start] = 0`

// Основной цикл алгоритма

ДЛЯ `count = 0` ДО `graph->vertices - 1`:

ЕСЛИ `stepByStep = true`:

 ВЫВЕСТИ "--- Шаг `count+1` ---"

 ВЫВЕСТИ "Текущие расстояния: "

 ДЛЯ КАЖДОЙ вершины `i`:

 ЕСЛИ `dist[i] = INF`: ВЫВЕСТИ "INF"

 ИНАЧЕ: ВЫВЕСТИ `dist[i]`

 ВЫВЕСТИ "Посещенные вершины: "

 ДЛЯ КАЖДОЙ вершины `i`:

 ЕСЛИ `visited[i] = true`: ВЫВЕСТИ `i`

 ЕСЛИ нет посещенных вершин: ВЫВЕСТИ "нет"

// Выбор вершины с минимальным расстоянием

`u = -1`

`minDist = INF`

ДЛЯ КАЖДОЙ вершины `v`:

```

    ЕСЛИ visited[v] = false И dist[v] < minDist:
        minDist = dist[v]
        u = v
ЕСЛИ u = -1:
    ПРЕРВАТЬ цикл
visited[u] = true
ЕСЛИ stepByStep = true:
    ВЫВЕСТИ "Выбираем вершину u (минимальное расстояние: dist[u])"
// Релаксация всех исходящих из u ребер
ДЛЯ КАЖДОЙ вершины v:
    weight = graph->matrix[u][v]
    ЕСЛИ weight != 0 И visited[v] = false:
        newDist = dist[u] + weight
        ЕСЛИ newDist < dist[v]:
            dist[v] = newDist
            parent[v] = u
            ЕСЛИ stepByStep = true:
                ВЫВЕСТИ "Проверяем переход u->v (вес weight): новое
расстояние newDist < старого dist[v] - ОБНОВЛЯЕМ"
            ИНАЧЕ ЕСЛИ stepByStep = true:
                ВЫВЕСТИ "Проверяем переход u->v (вес weight): новое
расстояние newDist >= старого dist[v] - не меняем"
        ЕСЛИ stepByStep = true:
            ВЫВЕСТИ "Нажмите Enter для продолжения..."
            ОЖИДАТЬ нажатия Enter
// Вывод результатов
ВЫЗВАТЬ printSolution(dist, parent, graph->vertices, start)
ВЫЗВАТЬ saveResultsToFile(graph, dist, parent, graph->vertices, start)
КОНЕЦ ФУНКЦИИ

```

Функция начинается с инициализации массивов: `dist` (расстояния), `visited` (посещенные вершины) и `parent` (предшественники). Все расстояния устанавливаются в бесконечность (`INF`), кроме расстояния до стартовой вершины, которое равно 0.

Затем выполняется основной цикл, который повторяется столько раз, сколько вершин в графе. На каждой итерации в пошаговом режиме выводится текущее состояние алгоритма: номера шага, текущие расстояния до всех вершин и список посещенных вершин.

Для выбора следующей вершины выполняется поиск непосещенной вершины с минимальным расстоянием. Если такой вершины нет (`u = -1`), цикл прерывается, так как все достижимые вершины уже обработаны.

Выбранная вершина `u` помечается как посещенная. Затем для каждого соседа `v` вершины `u` (то есть для каждой вершины, в которую ведет ребро из `u`) выполняется релаксация: если найден путь короче текущего, расстояние обновляется, а в массив предшественников записывается информация о вершине, через которую был найден этот путь.

В пошаговом режиме для каждого ребра выводится информация о проверке и результате: обновление расстояния или отказ от изменения. После каждого шага программа ожидает нажатия клавиши `Enter`.

После завершения алгоритма выводится таблица результатов с помощью функции `printSolution` и сохраняются данные в файл с помощью функции `saveResultsToFile`.

Функция `minDistance()` находит вершину с минимальным расстоянием среди непосещенных. Она перебирает все вершины графа и возвращает индекс вершины с минимальным значением `dist[]`, которая еще не была посещена. Если все вершины посещены или все непосещенные вершины имеют бесконечное расстояние, функция возвращает -1:

ФУНКЦИЯ `minDistance(dist, visited, vertices)`:

`min = INF`

`min_index = -1`

ДЛЯ КАЖДОЙ вершины v ОТ 0 ДО $vertices-1$:

ЕСЛИ $visited[v] = false$ И $dist[v] \leq min$:

$min = dist[v]$

$min_index = v$

ВЕРНУТЬ min_index

КОНЕЦ ФУНКЦИИ

Функция `printSolution()` форматирует и выводит итоговую таблицу результатов. Для каждой вершины отображается кратчайшее расстояние от стартовой вершины и сам путь в виде последовательности вершин, восстановленный с помощью массива предшественников. Для недостижимых вершин указывается соответствующая пометка:

ФУНКЦИЯ `printSolution(dist, parent, vertices, start)`:

ВЫВЕСТИ заголовок таблицы

ДЛЯ КАЖДОЙ вершины i ОТ 0 ДО $vertices-1$:

ЕСЛИ $dist[i] = INF$:

ВЫВЕСТИ " $| i |$ Нет $|$ Недостижима $|$ "

ИНАЧЕ:

// Восстановление пути

$path$ = пустой список

$current = i$

ПОКА $current \neq -1$:

 ДОБАВИТЬ $current$ В НАЧАЛО $path$

$current = parent[current]$

// Формирование строки пути

$path_str = ""$

ДЛЯ КАЖДОГО элемента j В $path$:

$path_str = path_str + j$

 ЕСЛИ j НЕ последний элемент:

$path_str = path_str + "->"$

// Вывод строки таблицы

ЕСЛИ $i = \text{start}$:

ВЫВЕСТИ "| i | $\text{dist}[i]$ | i |"

ИНАЧЕ:

ВЫВЕСТИ "| i | $\text{dist}[i]$ | path_str |"

// Выравнивание пробелами

ВЫВЕСТИ пробелы для выравнивания

ВЫВЕСТИ "|"

ВЫВЕСТИ нижнюю границу таблицы

КОНЕЦ ФУНКЦИИ

Функция `saveResultsToFile()` сохраняет все данные о графе и результатах работы алгоритма в текстовый файл. В файл записываются параметры графа, матрица смежности в читаемом формате и таблица кратчайших путей. Это позволяет пользователю проанализировать результаты позднее или использовать их для отчетности:

ФУНКЦИЯ `saveResultsToFile(graph, dist, parent, vertices, start)`:

ОТКРЫТЬ файл "`dijkstra_results.txt`" для записи

ЕСЛИ файл не удалось открыть:

ВЫВЕСТИ сообщение об ошибке

ВЕРНУТЬСЯ

// Запись заголовка и параметров

ЗАПИСАТЬ В ФАЙЛ "=== РЕЗУЛЬТАТЫ АЛГОРИТМА ДЕЙКСТРЫ
==="

ЗАПИСАТЬ В ФАЙЛ "Количество вершин: `vertices`"

ЗАПИСАТЬ В ФАЙЛ "Тип графа:
ориентированный/неориентированный"

ЗАПИСАТЬ В ФАЙЛ "Стартовая вершина: `start`"

// Запись матрицы смежности

ЗАПИСАТЬ В ФАЙЛ "Матрица смежности:"

ЗАПИСАТЬ В ФАЙЛ заголовок матрицы

ДЛЯ КАЖДОЙ строки i ОТ 0 ДО `vertices-1`:

```

    ЗАПИСАТЬ В ФАЙЛ номер строки i
    ДЛЯ КАЖДОГО столбца j ОТ 0 ДО vertices-1:
        ЕСЛИ graph->matrix[i][j] = 0:
            ЗАПИСАТЬ В ФАЙЛ " ."
        ИНАЧЕ:
            ЗАПИСАТЬ В ФАЙЛ graph->matrix[i][j]
            ЗАПИСАТЬ В ФАЙЛ перевод строки
        // Запись таблицы результатов
    ЗАПИСАТЬ В ФАЙЛ заголовок таблицы
    ДЛЯ КАЖДОЙ вершины i ОТ 0 ДО vertices-1:
        ЕСЛИ dist[i] = INF:
            ЗАПИСАТЬ В ФАЙЛ "| i | Нет | Недостижима |"
        ИНАЧЕ:
            // Восстановление пути для записи в файл
            path = пустой список
            current = i
            ПОКА current != -1:
                ДОБАВИТЬ current В НАЧАЛО path
                current = parent[current]
            // Формирование строки пути
            path_str = ""
            ДЛЯ КАЖДОГО элемента j В path:
                path_str = path_str + j
            ЕСЛИ j НЕ последний элемент:
                path_str = path_str + "->"
            // Запись строки в файл
            ЕСЛИ i = start:
                ЗАПИСАТЬ В ФАЙЛ "| i | dist[i] | i |"
            ИНАЧЕ:
                ЗАПИСАТЬ В ФАЙЛ "| i | dist[i] | path_str |"

```

```
// Выравнивание
ЗАПИСАТЬ В ФАЙЛ пробелы для выравнивания
ЗАПИСАТЬ В ФАЙЛ "|"
ЗАКРЫТЬ файлВЫВЕСТИ сообщение об успешном сохранении
КОНЕЦ ФУНКЦИИ
```


4. Тестирование

Тестирование проводилось как в процессе разработки, так и после её завершения. Каждая новая функциональность проверялась в режиме отладки для выявления и устранения ошибок на ранних этапах. В ходе тестирования были обнаружены и исправлены проблемы, связанные с обработкой граничных условий, корректностью генерации графа и работой алгоритма поиска кратчайших путей. Описание тестовых сценариев представлено в таблице 1.

Таблица 1 - Описание поведения программы при тестировании

№	Описание теста	Предусловие	Тестирование	Ожидаемый результат
1	Запуск программы	нет	Запуск программы с помощью VisualStudio	Корректный запуск программы, отображение главного меню с приветствием и запросом параметров графа.
2	Создание графа	Программа запущена, введен размер и выбран способ заполнения графа	Ввод количества вершин, выбор типа графа, разрешение/запрет петель и изолированных вершин	Создание графа в памяти, генерация матрицы смежности с учётом всех заданных параметров, вывод матрицы на экран.

Продолжение таблицы 1

№	Описание теста	Предусловие	Тестирование	Ожидаемый результат
3	Просмотр матрицы смежности	Граф успешно создан	Автоматический вывод после генерации или выбор соответствующей опции в меню	Чёткое отображение матрицы смежности с весами рёбер, нули отмечены точками, заголовки строк и столбцов содержат номера вершин.
4	Выполнение алгоритма Дейкстры	Граф создан, матрица смежности отображена	Ввод стартовой вершины, запуск алгоритма в пошаговом или итоговом режиме	Корректное выполнение алгоритма, вывод таблицы кратчайших расстояний и путей от стартовой вершины до всех остальных.
5	Сохранение результатов в файл	Алгоритм успешно выполнен, результаты выведены на экран	Автоматическое сохранение или подтверждение пользователя	Создание файла <code>dijkstra_results.txt</code> с полной информацией о графе и результатах работы алгоритма.
6	Обработка изолированных вершин	Задан запрет на изолированные вершины, граф сгенерирован с их наличием	Автоматическая проверка функцией <code>checkIsolatedVertices()</code>	Вывод предупреждения об обнаружении изолированных вершин и их автоматическое устранение путём добавления случайных рёбер.

Продолжение таблицы 1

№	Описание теста	Предусловие	Тестирование	Ожидаемый результат
7	Пошаговая демонстрация алгоритма	Количество вершин графа ≤ 10	Запуск алгоритма с флагом <code>stepByStep = 1</code>	Поэтапный вывод состояния алгоритма: текущие расстояния, посещённые вершины, выбор следующей вершины, релаксация рёбер, пауза после каждого шага.
8	Пакетный режим работы	Количество вершин графа > 10	Запуск алгоритма с флагом <code>stepByStep = 0</code>	Вывод только итоговой таблицы результатов без промежуточных шагов.
9	Повторный запуск программы	Завершён один цикл работы программы	Выбор опции повторного выполнения при запросе	Возврат в главное меню, возможность задать новые параметры графа и выполнить алгоритм снова.
10	Обработка некорректного ввода	Программа ожидает ввода числовых значений	Ввод символов, отрицательных чисел, значений вне допустимого диапазона	Вывод сообщения об ошибке, повторный запрос корректных данных, продолжение работы программы без сбоев.

Ниже будет приведено несколько тестов.

4.1. Создание графа (последовательности)

В главном меню выбираю из двух типов графа (Рисунок 10).

```
=== РЕАЛИЗАЦИЯ АЛГОРИТМА ДЕЙКСТРЫ ===  
  
Введите количество вершин (1-100): 5  
  
Тип графа:  
1. Ориентированный  
2. Неориентированный  
Выберите (1-2): 2
```

Рисунок 10 – Меню выбора перед выполнением

Затем выводятся две строки с возможностью появления петель и/или изолированных вершин (Рисунок 11).

```
Разрешить петли у вершин? (0-нет, 1-да): 1  
  
Разрешить изолированные вершины? (0-нет, 1-да): 1
```

Рисунок 11 – Условия появления петель и изолированных вершин

После чего в консоли отображается сгенерированная матрица смежности, соответствующая ранее указанным параметрам. Тест пройден успешно.

4.2. Просмотр созданного графа (последовательности)

После того, как мы задали параметры для графа программа, выводит матрицу смежности с предупреждением о том, нет ли случайных изолированных вершин, если мы их запретили (Рисунок 12).

Изолированных вершин не обнаружено.

Матрица смежности:

	0	1	2	3	4
0	.	.	.	5	.
1	.	.	3	.	.
2	.	3	8	10	.
3	5	.	10	.	7
4	.	.	.	7	8

Рисунок 12 – Матрица смежности с отслеживанием

В зависимости от количества вершин придет сообщение о том, будет ли пошаговая реализация или нет (Рисунок 13).

Количество вершин <= 10, будет выполнена пошаговая демонстрация
Нажимайте Enter после каждого шага для продолжения...

Рисунок 13 – Сообщение о выполнении пошаговой реализации

Наблюдаемая нами матрица смежности соответствует введенными параметрами ранее. Тест пройден успешно.

4.3. Анализ созданного графа (последовательности)

После генерации смежности выводится строка с выбором, с какой вершины начинать алгоритм и как было описано ранее, выводится сообщение о возможности выполнения пошаговой реализации в зависимости от количества вершин (Рисунок 14).

```

Введите стартовую вершину (0-4): 0
Количество вершин <= 10, будет выполнена пошаговая демонстрация
Нажимайте Enter после каждого шага для продолжения...

```

Рисунок 14 – Строка ввода стартовой вершины с сообщением

В данном случае будет подробная пошаговая реализация. (Рисунок 15).

```

=== НАЧАЛО АЛГОРИТМА ДЕЙКСТРА (стартовая вершина: 0) ===

--- Шаг 1 ---
Текущие расстояния:  0 INF INF INF INF
Посещенные вершины: 0
Выбираем вершину 0 (минимальное расстояние: 0)
  Вершина 0 уже посещена
  Переход 0->1: ребра нет
  Переход 0->2: ребра нет
  Проверяем переход 0->3 (вес 5): новое расстояние 5 < INF - ОБНОВЛЯЕМ
  Переход 0->4: ребра нет

Нажмите Enter для продолжения...

--- Шаг 2 ---
Текущие расстояния:  0 INF INF 5 INF
Посещенные вершины: 0
Выбираем вершину 3 (минимальное расстояние: 5)
  Вершина 0 уже посещена
  Переход 3->1: ребра нет
  Проверяем переход 3->2 (вес 10): новое расстояние 15 < INF - ОБНОВЛЯЕМ
  Вершина 3 уже посещена
  Проверяем переход 3->4 (вес 7): новое расстояние 12 < INF - ОБНОВЛЯЕМ

Нажмите Enter для продолжения...

--- Шаг 3 ---
Текущие расстояния:  0 INF 15 5 12
Посещенные вершины: 0 3
Выбираем вершину 4 (минимальное расстояние: 12)
  Вершина 0 уже посещена
  Переход 4->1: ребра нет
  Переход 4->2: ребра нет
  Вершина 3 уже посещена
  Вершина 4 уже посещена

Нажмите Enter для продолжения...

--- Шаг 4 ---
Текущие расстояния:  0 INF 15 5 12
Посещенные вершины: 0 3 4
Выбираем вершину 2 (минимальное расстояние: 15)
  Вершина 0 уже посещена
  Проверяем переход 2->1 (вес 3): новое расстояние 18 < INF - ОБНОВЛЯЕМ
  Вершина 2 уже посещена
  Вершина 3 уже посещена
  Вершина 4 уже посещена

Нажмите Enter для продолжения...

--- Шаг 5 ---
Текущие расстояния:  0 18 15 5 12
Посещенные вершины: 0 2 3 4
Выбираем вершину 1 (минимальное расстояние: 18)
  Вершина 0 уже посещена
  Вершина 1 уже посещена
  Вершина 2 уже посещена
  Вершина 3 уже посещена
  Вершина 4 уже посещена

Нажмите Enter для продолжения...

```

Рисунок 15 – Пошаговая реализация алгоритма

Мы можем наблюдать подробное описание выполнения Дейкстра на текущей последовательности со всеми найденными вершинами и их текущих расстояний. Тест пройден успешно.

После того, как была проведена пошаговая реализация, выводится итоговая таблица с результатами пройденного алгоритма (Рисунок 16).

Вершина	Расстояние	Путь
0	0	0
1	18	0->3->2->1
2	15	0->3->2
3	5	0->3
4	12	0->3->4

Рисунок 16 – Итоговая таблица

Если же была задана матрица, количество которой превышает данного минимума, то пошаговая реализация будет пропущена и выведет итоговый результат сразу после предупреждения (Рисунок 17).

Количество вершин > 10, выводится только итоговая таблица

Рисунок 17 – Сообщение о выполнении пакетного набора данных

4.4. Сохранение результатов

Сразу после выполнения анализа последовательности результат автоматически сохраняется в текстовый файл. Но, нам также дается выбор выполнить ещё одну генерацию, которая возвращается обратно в меню создания новой матрицы смежности, но в данном случае прошлый результат будет стерт и сохранится новый (Рисунок 18).

Результаты сохранены в файл 'dijkstra_results.txt'
Хотите выполнить еще одну генерацию? (1 - да, 0 - нет):

Рисунок 18 – Сообщение о сохранении итогового результата в текстовый файл

После чего текстовый файл должен появиться в папке проекта (Рисунок 19).

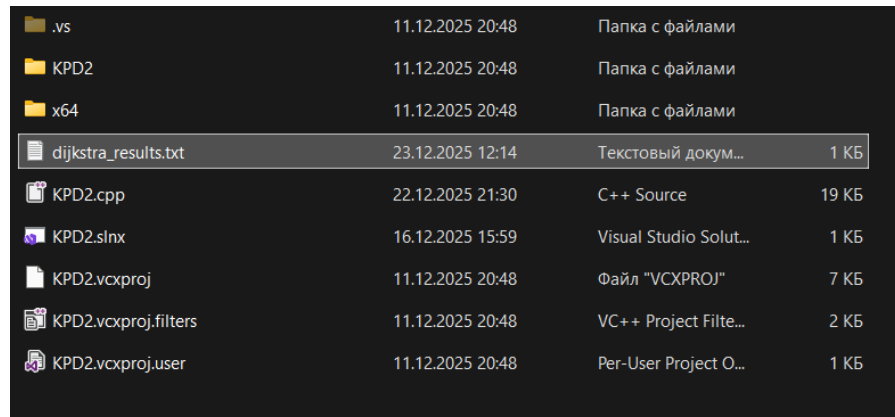


Рисунок 19 – Папка проекта с сохраненным текстовым файлом

Для проверки успешного сохранения откроем созданный файл (Рисунок 20).

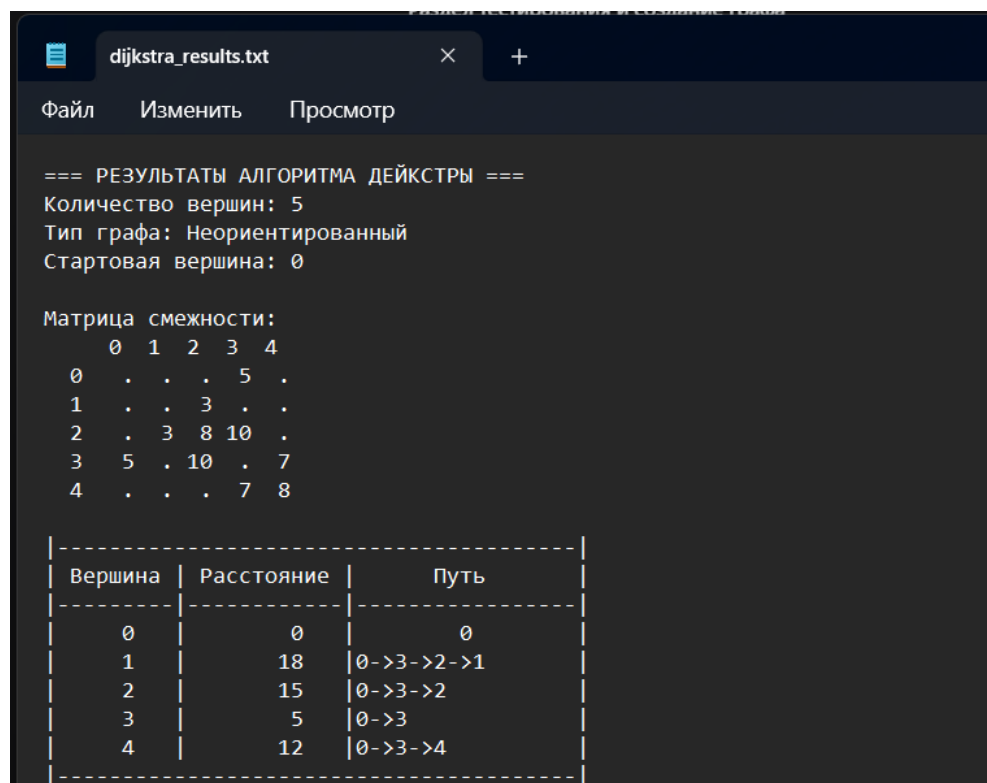


Рисунок 20 – Текстовый файл результата

Все результаты были сохранены. Тест успешно пройден.

4.5. Результаты

Результаты тестирования программы приведены в таблице 2.

Таблица 2 - Результаты поведения программы при тестировании

№	Описание теста	Полученный результат
1	Запуск программы	Верно
2	Создание графа	Верно
3	Просмотр матрицы смежности	Верно
4	Выполнение алгоритма Дейкстра	Верно
5	Сохранение результатов	Верно
6	Обработка изолированных вершин	Верно
7	Пошаговая демонстрация алгоритма (≤ 10)	Верно
8	Пакетный режим работы (> 10)	Верно
9	Повторный запуск программы	Верно
10	Обработка некорректного ввода	Верно

Все тесты выполнены успешно, программа работает корректно и соответствует предъявленным требованиям. Интерфейс интуитивно понятен, все режимы функционируют как ожидалось, обработка ошибок осуществляется корректно.

5. Ручной расчет

Рассмотрим ориентированный взвешенный граф из 4 вершин, без петель и изолированных вершин, представленный матрицей смежности (рисунок).

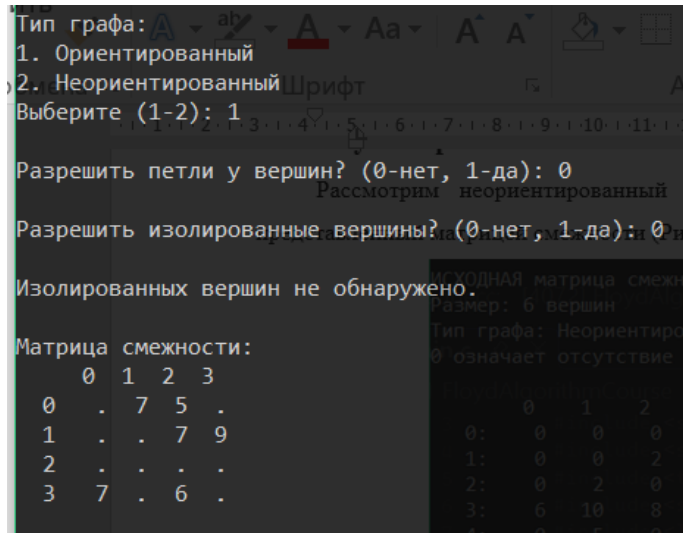


Рисунок 21 – Матрица смежности

Условные обозначения:

Точка («.») на диагонали и вне диагонали – расстояние от вершины самой себя и отсутствие прямого пути между вершинами;

Число >0 – вес ребра между вершинами

5.1. Пошаговый расчет алгоритма

Дано:

Ориентированный граф из 4 вершин (0-3).

Матрица смежности:

Из/В	0	1	2	3
0	-	7	5	-
1	-	-	7	9
2	-	-	-	-
3	7	-	6	-

Задача:

Найти кратчайшее расстояние от вершины 0 до всех остальных.

Шаг 1: Инициализация

- $\text{dist}[0] = 0$, остальные $= \infty$
- Необработанные: $\{0, 1, 2, 3\}$

Вершина 0 1 2 3

Расстояние 0 ∞ ∞ ∞

Шаг 2: Обрабатываем вершину 0

- Текущая вершина: 0 ($\text{dist} = 0$)
- Смотрим её рёбра:
 - $0 \rightarrow 1$ вес 7: $0 + 7 = 7 < \infty \rightarrow \text{dist}[1] = 7$
 - $0 \rightarrow 2$ вес 5: $0 + 5 = 5 < \infty \rightarrow \text{dist}[2] = 5$
- Переносим 0 в обработанные.

Вершина 0 1 2 3

Расстояние 0 7 5 ∞

Шаг 3: Выбираем следующую вершину с $\min \text{dist}$

Необработанные: $\{1, 2, 3\}$.

Минимум: вершина 2 ($\text{dist} = 5$).

- У вершины 2 исходящих рёбер нет $\rightarrow$ ничего не меняем
- Переносим 2 в обработанные.

Вершина 0 1 2 3

Расстояние 0 7 5 ∞

Шаг 4: Выбираем следующую вершину с $\min \text{dist}$

Необработанные: $\{1, 3\}$.

Минимум: вершина 1 ($\text{dist} = 7$).

- Рёбра из 1: $1 \rightarrow 2$ вес 7: $7 + 7 = 14 > 5 \rightarrow$ не меняем
- $1 \rightarrow 3$ вес 9: $7 + 9 = 16 < \infty \rightarrow \text{dist}[3] = 16$
- Переносим 1 в обработанные.

Вершина 0 1 2 3

Расстояние 0 7 5 16

Шаг 5: Обрабатываем последнюю вершину 3

Необработанные: $\{3\}$.

- Рёбра из 3:

- $3 \rightarrow 0$ вес 7: $16 + 7 = 23 > 0 \rightarrow$ не меняем
- $3 \rightarrow 2$ вес 6: $16 + 6 = 22 > 5 \rightarrow$ не меняем
- Переносим 3 в обработанные.

Итог:

Конечная вершина Кратчайшее расстояние от 0 Путь

0 0 0

1 7 0 $\rightarrow$ 1

2 5 0 $\rightarrow$ 2

3 16 0 $\rightarrow$ 1 $\rightarrow$ 3

Расчёт завершён.

Заключение

В ходе выполнения данной курсовой работы были получены практические навыки в области разработки алгоритмов на графах и реализации алгоритма Дейкстры для поиска кратчайших путей. Была успешно выполнена поставленная задача по созданию программы, которая позволяет генерировать взвешенные графы с различными параметрами, визуализировать их структуру и находить кратчайшие пути от заданной начальной вершины до всех остальных.

Основным результатом работы является консольное приложение, реализующее алгоритм Дейкстры для взвешенных графов с неотрицательными весами ребер. Программа поддерживает работу как с ориентированными, так и с неориентированными графами, предоставляет возможности настройки параметров генерации (разрешение петель и изолированных вершин), а также включает два режима работы: пошаговая демонстрация для графов небольшого размера и пакетная обработка для графов большего размера.

Тестирование программы подтвердило корректность работы алгоритма для различных типов графов и входных данных. Программа успешно обрабатывает графы разной плотности, корректно определяет недостижимые вершины и строит кратчайшие пути для достижимых вершин.

Разработанная программа представляет собой законченное решение для демонстрации и применения алгоритма Дейкстры, сочетающее в себе функциональность генерации графов, наглядного представления результатов и сохранения данных для последующего анализа. Программа имеет интуитивно понятный интерфейс и может быть использована как для учебных целей при изучении алгоритмов на графах, так и для решения практических задач поиска кратчайших путей в различных приложениях.

Список литературы

1. Дейкстра, Э. В. Заметка о двух проблемах в связи с графами / Э. В. Дейкстра // *Numerische Mathematik*. — 1959. — Vol. 1. — P. 269–271
2. Кормен, Т., Лейзерсон, Ч., Ривест, Р., Штайн, К. Алгоритмы: построение и анализ. — 3-е изд. — М. : Вильямс, 2013. — 1328 с.
3. Свами, М., Тхуласираман, К. Графы, сети и алгоритмы / М. Свами, К. Тхуласираман. — М. : Мир, 1984. — 454 с.
4. Асанов, М. О., Баранский, В. А., Расин, В. В. Дискретная математика: графы, матроиды, алгоритмы. — СПб. : Лань, 2010. — 368 с.
5. Скиена, С. С. Алгоритмы. Руководство по разработке. — 2-е изд. — СПб. : БХВ-Петербург, 2011. — 720 с.

Приложение А.

Листинг программы.

```
#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h>

#include <stdlib.h>

#include <time.h>

#include <limits.h>

#include <locale.h>

#define MAX_VERTICES 100

#define INF INT_MAX

typedef struct {

    int** matrix;

    int vertices;

    int directed;

    int loops;

    int isolated;

} Graph;

Graph* createGraph(int vertices, int directed, int loops, int isolated) {

    Graph* graph = (Graph*)malloc(sizeof(Graph));

    graph->vertices = vertices;

    graph->directed = directed;

    graph->loops = loops;

    graph->isolated = isolated;

    graph->matrix = (int**)malloc(vertices * sizeof(int*));
```

```

for (int i = 0; i < vertices; i++) {

    graph->matrix[i] = (int*)malloc(vertices * sizeof(int));

    for (int j = 0; j < vertices; j++) {

        graph->matrix[i][j] = 0;

    }

}

return graph;

}

void checkIsolatedVertices(Graph* graph) {

    int isolatedCount = 0;

    for (int i = 0; i < graph->vertices; i++) {

        int hasEdges = 0;

        for (int j = 0; j < graph->vertices; j++) {

            if (i == j && graph->loops && graph->matrix[i][j] != 0) {

                hasEdges = 1;

                break;

            }

            if (i != j) {

                if (graph->directed) {

                    if (graph->matrix[i][j] != 0 || graph->matrix[j][i] != 0) {

                        hasEdges = 1;

                        break;

                    }

                }

            }

            else {

```



```

        if (graph->matrix[i][j] != 0) {

            hasEdges = 1;

            break;

        }

    }

}

if (!hasEdges) {

    isolatedCount++;

}

}

if (isolatedCount > 0) {

    printf("\nОбнаружено изолированных вершин: %d\n", isolatedCount);

    printf("Изолированные вершины: ");

    for (int i = 0; i < graph->vertices; i++) {

        int hasEdges = 0;

        for (int j = 0; j < graph->vertices; j++) {

            if (i == j && graph->loops && graph->matrix[i][j] != 0) {

                hasEdges = 1;

                break;

            }

            if (i != j) {

                if (graph->directed) {

                    if (graph->matrix[i][j] != 0 || graph->matrix[j][i] != 0) {

                        hasEdges = 1;


```

```

        break;

    }

}

else {

    if (graph->matrix[i][j] != 0) {

        hasEdges = 1;

        break;

    }

}

}

}

if (!hasEdges) {

    printf("%d ", i);

}

}

printf("\n");

if (!graph->isolated && isolatedCount > 0) {

    printf("ВНИМАНИЕ: Изолированные вершины обнаружены, хотя вы
запретили их!\n");

    printf("Автоматически добавляем связи для устранения
изолированных вершин...\n");

    for (int i = 0; i < graph->vertices; i++) {

        int hasEdges = 0;

        for (int j = 0; j < graph->vertices; j++) {

```

```

if (i == j && graph->loops && graph->matrix[i][j] != 0) {

    hasEdges = 1;

    break;

}

if (i != j) {

    if (graph->directed) {

        if (graph->matrix[i][j] != 0 || graph->matrix[j][i] != 0) {

            hasEdges = 1;

            break;

        }

    }

    else {

        if (graph->matrix[i][j] != 0) {

            hasEdges = 1;

            break;

        }

    }

}

if (!hasEdges) {

    int j;

    do {

        j = rand() % graph->vertices;

    } while (j == i);

    int weight = 1 + rand() % 10;

```

```

graph->matrix[i][j] = weight;

if (!graph->directed) {

    graph->matrix[j][i] = weight;

}

printf(" Добавлено ребро %d-%d (вес: %d)\n", i, j, weight);

}

}

printf("Изолированные вершины устранены.\n");

}

}

else {

    printf("\nИзолированных вершин не обнаружено.\n");

}

}

void generateGraph(Graph* graph) {

    srand(time(NULL));

    for (int i = 0; i < graph->vertices; i++) {

        for (int j = 0; j < graph->vertices; j++) {

            if (i == j) {

                if (graph->loops && (rand() % 100 < 25)) {

                    graph->matrix[i][j] = 1 + rand() % 10;

                }

            }

            else {

                graph->matrix[i][j] = 0;

```

```

    }

    continue;

}

if (!graph->directed && i > j) {

    continue;

}

int isolated_chance = graph->isolated ? 10 : 0;

if (rand() % 100 < isolated_chance) {

    graph->matrix[i][j] = 0;

}

else {

    if (rand() % 100 < 40) {

        graph->matrix[i][j] = 1 + rand() % 10;

        if (!graph->directed) {

            graph->matrix[j][i] = graph->matrix[i][j];

        }

    }

    else {

        graph->matrix[i][j] = 0;

    }

}

}

checkIsolatedVertices(graph);

}

```

```

void printMatrix(Graph* graph) {

    printf("\nМатрица смежности:\n");

    printf(" ");

    for (int i = 0; i < graph->vertices; i++) {

        printf("%3d", i);

    }

    printf("\n");

    for (int i = 0; i < graph->vertices; i++) {

        printf("%3d ", i);

        for (int j = 0; j < graph->vertices; j++) {

            if (graph->matrix[i][j] == 0) {

                printf(" .");

            }

            else {

                printf("%3d", graph->matrix[i][j]);

            }

        }

        printf("\n");

    }

    printf("\n");

}

int minDistance(int dist[], int visited[], int vertices) {

    int min = INF;

    int min_index = -1;

    for (int v = 0; v < vertices; v++) {

```

```

    if (!visited[v] && dist[v] <= min) {

        min = dist[v];

        min_index = v;

    }

}

return min_index;

}

void printSolution(int dist[], int parent[], int vertices, int start) {

    printf("\n|-----|");

    printf("\n| Вершина | Расстояние | Путь |");

    printf("\n|-----|-----|-----|\n");

    for (int i = 0; i < vertices; i++) {

        if (dist[i] == INF) {

            printf("| %3d | %6s | Недостижима | \n", i, "Нет");

        }

        else {

            int path[MAX_VERTICES];

            int count = 0;

            int current = i;

            while (current != -1) {

                path[count++] = current;

                current = parent[current];

            }

            printf("| %3d | %6d |", i, dist[i]);

            if (i == start) {

```

```

        printf("    %3d    |\\n", i);

    }

    else {

        for (int j = count - 1; j >= 0; j--) {

            printf("%d", path[j]);

            if (j > 0) printf("->");

        }

        int path_length = 0;

        for (int j = count - 1; j >= 0; j--) {

            path_length++;

            if (path[j] >= 10) path_length++;

            if (path[j] >= 100) path_length++;

        }

        path_length += (count - 1) * 2;

        int spaces = 15 - path_length;

        if (spaces < 0) spaces = 0;

        for (int s = 0; s < spaces; s++) printf(" ");

        printf(" |\\n");

    }

}

}

printf("-----\\n");

}

void saveResultsToFile(Graph* graph, int dist[], int parent[], int vertices, int start) {

    FILE* file = fopen("dijkstra_results.txt", "w");

```



```

if (file == NULL) {

    printf("Ошибка при создании файла для сохранения результатов!\n");

    return;

}

fprintf(file, "=== РЕЗУЛЬТАТЫ АЛГОРИТМА ДЕЙКСТРЫ ===\n");

fprintf(file, "Количество вершин: %d\n", vertices);

fprintf(file, "Тип графа: %s\n", graph->directed ? "Ориентированный" :
"Неориентированный");

fprintf(file, "Стартовая вершина: %d\n\n", start);

fprintf(file, "Матрица смежности:\n");

fprintf(file, " ");

for (int i = 0; i < vertices; i++) {

    fprintf(file, "%3d", i);

}

fprintf(file, "\n");

for (int i = 0; i < vertices; i++) {

    fprintf(file, "%3d ", i);

    for (int j = 0; j < vertices; j++) {

        if (graph->matrix[i][j] == 0) {

            fprintf(file, " .");

        }

        else {

            fprintf(file, "%3d", graph->matrix[i][j]);

        }

    }

}

```

```

    }

    fprintf(file, "\n");

}

fprintf(file, "\n");

fprintf(file, "|-----|\n");

fprintf(file, "| Вершина | Расстояние | Путь |\n");

fprintf(file, "|-----|-----|-----|\n");

for (int i = 0; i < vertices; i++) {

    if (dist[i] == INF) {

        fprintf(file, "| %3d | %6s | Недостижима |\n", i, "Нет");

    }

    else {

        int path[MAX_VERTICES];

        int count = 0;

        int current = i;

        while (current != -1) {

            path[count++] = current;

            current = parent[current];

        }

        fprintf(file, "| %3d | %6d |", i, dist[i]);

        if (i == start) {

            fprintf(file, " %3d |\n", i);

        }

        else {

            for (int j = count - 1; j >= 0; j--) {

```

```

        fprintf(file, "%d", path[j]);

        if (j > 0) fprintf(file, "->");

    }

    int path_length = 0;

    for (int j = count - 1; j >= 0; j--) {

        path_length++;

        if (path[j] >= 10) path_length++;

        if (path[j] >= 100) path_length++;

    }

    path_length += (count - 1) * 2;

    int spaces = 15 - path_length;

    if (spaces < 0) spaces = 0;

    for (int s = 0; s < spaces; s++) fprintf(file, " ");

    fprintf(file, " |\\n");

    }

    }

}

fprintf(file, "|-----|\\n");

fclose(file);

printf("\\nРезультаты сохранены в файл 'dijkstra_results.txt'\\n");

}

void dijkstra(Graph* graph, int start, int stepByStep) {

    int vertices = graph->vertices;

    int dist[MAX_VERTICES];

    int visited[MAX_VERTICES] = { 0 };

```

```

int parent[MAX_VERTICES];

for (int i = 0; i < vertices; i++) {

    dist[i] = INF;

    parent[i] = -1;

}

dist[start] = 0;

printf("\n=== НАЧАЛО АЛГОРИТМА ДЕЙКСТРЫ (стартовая вершина: %d) ===\n",
start);

for (int count = 0; count < vertices; count++) {

    if (stepByStep) {

        printf("\n--- Шаг %d ---\n", count + 1);

        printf("Текущие расстояния: ");

        for (int i = 0; i < vertices; i++) {

            if (dist[i] == INF) printf(" INF ");

            else printf("%4d ", dist[i]);

        }

        printf("\nПосещенные вершины: ");

        int visited_count = 0;

        for (int i = 0; i < vertices; i++) {

            if (visited[i]) {

                printf("%d ", i);

                visited_count++;

            }

        }

    }

}

```

```

        if (visited_count == 0) printf("Нет");

        printf("\n");

    }

    int u = minDistance(dist, visited, vertices);

    if (u == -1) {

        if (stepByStep) printf("Все достижимые вершины обработаны!\n");

        break;

    }

    visited[u] = 1;

    if (stepByStep) {

        if (dist[u] == INF) {

            printf("Выбираем вершину %d (недостижима, расстояние: INF)\n", u);

            printf("Оставшиеся вершины недостижимы из начальной
вершины\n");

            break;

        }

        else {

            printf("Выбираем вершину %d (минимальное расстояние: %d)\n", u,
dist[u]);

        }

    }

    if (dist[u] == INF) {

        if (stepByStep) printf("Недостижимые вершины остались, обработка
завершена\n");

```

```

        break;

    }

    for (int v = 0; v < vertices; v++) {

        if (!visited[v] && graph->matrix[u][v] != 0 && dist[u] != INF) {

            int newDist = dist[u] + graph->matrix[u][v];

            if (stepByStep) {

                printf(" Проверяем переход %d->%d (вес %d): ", u, v, graph->matrix[u][v]);

                if (dist[v] == INF) {

                    printf("новое расстояние %d < INF - ОБНОВЛЯЕМ\n", newDist);

                }

                else if (newDist < dist[v]) {

                    printf("новое расстояние %d < старого %d - ОБНОВЛЯЕМ\n", newDist,

dist[v]);

                }

                else {

                    printf("новое расстояние %d >= старого %d - не меняем\n", newDist,

dist[v]);

                }

            }

            if (newDist < dist[v]) {

                dist[v] = newDist;

                parent[v] = u;

            }

        }

    }

```

```

else if (stepByStep && !visited[v] && graph->matrix[u][v] == 0) {

    printf(" Переход %d->%d: ребра нет\n", u, v);

}

else if (stepByStep && visited[v]) {

    printf(" Вершина %d уже посещена\n", v);

}

}

if (stepByStep) {

    printf("\nНажмите Enter для продолжения...");

    getchar();

}

}

printf("\n=== РЕЗУЛЬТАТЫ АЛГОРИТМА ДЕЙКСТРЫ ===\n");

printSolution(dist, parent, graph->vertices, start);

saveResultsToFile(graph, dist, parent, graph->vertices, start);

}

void runAlgorithm() {

    int vertices, directed, loops, isolated;

    int startVertex;

    char choice;

    do {

        system("cls");

        printf("=== РЕАЛИЗАЦИЯ АЛГОРИТМА ДЕЙКСТРЫ ===\n");

        do {

```

```

printf("\nВведите количество вершин (1-100): ");

scanf("%d", &vertices);

if (vertices < 1 || vertices > 100) {

    printf("Ошибка! Введите число от 1 до 100\n");

}

} while (vertices < 1 || vertices > 100);

printf("\nТип графа:");

printf("\n1. Ориентированный");

printf("\n2. Неориентированный");

printf("\nВыберите (1-2): ");

scanf("%d", &directed);

directed = (directed == 1);

printf("\nРазрешить петли у вершин? (0-нет, 1-да): ");

scanf("%d", &loops);

printf("\nРазрешить изолированные вершины? (0-нет, 1-да): ");

scanf("%d", &isolated);

Graph* graph = createGraph(vertices, directed, loops, isolated);

generateGraph(graph);

printMatrix(graph);

do {

    printf("\nВведите стартовую вершину (0-%d): ", vertices - 1);

    scanf("%d", &startVertex);

    if (startVertex < 0 || startVertex >= vertices) {

        printf("Ошибка! Неверный номер вершины\n");

    }

} while (startVertex < 0 || startVertex >= vertices);

```



```

    }

    } while (startVertex < 0 || startVertex >= vertices);

    int stepByStep = (vertices <= 10) ? 1 : 0;

    if (stepByStep) {

        printf("\nКоличество вершин <= 10, будет выполнена пошаговая  

демонстрация\n");

        printf("\nНажимайте Enter после каждого шага для продолжения...\n");

        getchar();

        getchar();

    }

    else {

        printf("\nКоличество вершин > 10, выводится только итоговая  

таблица\n");

    }

    dijkstra(graph, startVertex, stepByStep);

    for (int i = 0; i < vertices; i++) {

        free(graph->matrix[i]);

    }

    free(graph->matrix);

    free(graph);

    printf("\nХотите выполнить еще одну генерацию? (1 - да, 0 - нет): ");

    scanf(" %c", &choice);

    getchar();

} while (choice == '1' || choice == 'y' || choice == 'Y' || choice == 'д' || choice == 'Д');

```

```
}  
  
int main() {  
  
    setlocale(LC_ALL, "Russian");  
  
    runAlgorithm();  
  
    printf("\nНажмите Enter для выхода...");  
  
    getchar();  
  
    return 0;  
  
}
```